

基于 DDPG 的双足机器人行走控制实验报告

姓名：邓一骏 学号：25SD36037

2026 年 6 月 5 日

目录

1	实验目标	3
2	环境与问题描述	3
2.1	任务背景	3
2.2	核心难点	4
3	算法原理	4
3.1	DDPG 算法概述	4
3.2	Actor-Critic 架构	4
3.3	训练机制	5
3.4	探索噪声与稳定性增强	5
4	实验设置	7
4.1	软硬件环境	7
4.2	超参数配置	7
4.3	网络结构细节	8
5	实验结果	8
5.1	训练过程总览	8
5.2	训练回报曲线	9
5.3	分阶段分析	9
5.4	关键里程碑事件	10
5.5	Loss 变化分析	10
6	结果分析	10
6.1	学习过程的时间演化规律	10
6.2	算法组件的有效性分析	11
6.3	训练高方差现象分析	12

目录	2
7 工程问题剖析与解决对策	13
8 结论与展望	13
8.1 实验结论	13
8.2 未来工作展望	14

1 实验目标

本实验以 OpenAI Gymnasium 中的双足机器人行走任务 BipedalWalker-v3 为研究对象，采用深度确定性策略梯度算法 (Deep Deterministic Policy Gradient, DDPG)，实现智能体对连续动作空间下机器人行走策略的学习。

核心研究问题： 如何在连续动作空间中，通过 Actor-Critic 结构、经验回放与目标网络机制，使双足机器人学会稳定、高效的行走策略，并在合理训练预算内达到环境求解标准 (评估平均回报 ≥ 300)。

量化评价指标：

- **训练累计回报** (Train Reward)：每个 episode 内所有即时奖励之和。
- **平滑回报** (Smoothed Reward)：取窗口 $K = 10$ 的滑动平均，用于消除噪声反映趋势。
- **评估平均回报** (Eval Reward)：每 20 个训练回合，在无探索噪声条件下运行 5 个 episode 的平均回报。
- **求解标准**：评估平均回报 ≥ 300 视为环境解决。

2 环境与问题描述

2.1 任务背景

BipedalWalker-v3 是 Gymnasium 框架中标准的连续控制测试环境，要求智能体控制一个双足机器人在起伏地形上前进。该环境具有如下技术特征：

- **观测空间** (Observation Space)：24 维连续向量，包含机体姿态角、角速度、关节角度与角速度、足底接触状态及前方地形 LIDAR 测距信息。
- **动作空间** (Action Space)：4 维连续向量，每个维度取值 $[-1, 1]$ ，对应四个关节电机的归一化扭矩输出。
- **奖励函数**：由前进速度奖励、能耗惩罚和姿态稳定性惩罚三部分构成。每前进一步获得正向奖励，电机过度使用产生负向惩罚，机体倾斜过大或跌倒时给予强负奖励并终止回合。
- **终止条件**：机器人躯干触地、头部高度过低、达到最大步数 (1600 步)、或成功到达终点。

2.2 核心难点

1. **连续动作空间**: 无法通过离散动作枚举进行 Q-Learning, 必须使用确定性策略梯度方法或随机策略方法。
2. **高维连续状态**: 24 维观测空间需要神经网络具备有效的特征提取能力。
3. **探索与利用的平衡**: 随机探索过强容易导致频繁摔倒, 探索过弱则无法发现有效行走策略。
4. **训练稳定性**: Actor 与 Critic 同时更新易导致值函数高估、梯度震荡和目标不匹配问题。
5. **奖励稀疏性与延迟性**: 初期大量负奖励掩盖了策略改进的信号, 需要较长的训练周期才能观察到正向回报。

3 算法原理

3.1 DDPG 算法概述

DDPG[1] 是一种面向连续动作空间的 off-policy Actor-Critic 算法。它将深度 Q 网络 (DQN) 的经验回放与目标网络思想扩展到连续控制领域, 结合确定性策略梯度 (Deterministic Policy Gradient, DPG) [2] 实现对连续动作的直接优化。

将双足机器人控制问题建模为马尔可夫决策过程 (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

其中状态空间 $\mathcal{S} \subset \mathbb{R}^{24}$, 动作空间 $\mathcal{A} \subset \mathbb{R}^4$, $\gamma = 0.99$ 为折扣因子。目标是学习确定性策略 $\mu_\theta: \mathcal{S} \rightarrow \mathcal{A}$ 最大化期望累计折扣回报:

$$J(\theta) = \mathbb{E}_{s_0 \sim \rho_0} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \mu_\theta(s_t)) \right]$$

3.2 Actor-Critic 架构

Actor 网络 (策略网络) Actor $\mu(s|\theta^\mu)$ 将状态映射到确定性动作, 采用双隐层全连接结构:

$$a_t = \tanh(h_2(W_2 \cdot \text{ReLU}(W_1 s + b_1) + b_2)) \cdot a_{\max} \quad (1)$$

其中 $W_1 \in \mathbb{R}^{400 \times 24}$, $W_2 \in \mathbb{R}^{300 \times 400}$, 输出层使用 $\tanh$ 将动作约束在 $[-1, 1]$ 后乘以动作上界 $a_{\max}$ 。

Critic 网络 (价值网络) Critic $Q(s, a|\theta^Q)$ 评估状态-动作对的价值, 将状态与动作拼接后输入:

$$Q(s, a) = W_4 \cdot \text{ReLU}(W_3 \cdot [s, a] + b_3) + b_4 \quad (2)$$

其中 $W_3 \in \mathbb{R}^{400 \times 28}$, $W_4 \in \mathbb{R}^{1 \times 300}$, 输出为标量 Q 值。

3.3 训练机制

经验回放 采用经验回放池 $\mathcal{D}$ 存储转移样本 $(s_t, a_t, r_t, s_{t+1}, d_t)$ 。训练时随机采样 N 个样本, 打破时间相关性并提高样本利用率。

目标网络与软更新 为 Actor 和 Critic 分别维护目标网络 μ' 和 Q' , 参数通过 Polyak 平均软更新:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}, \quad \theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \quad (3)$$

其中 $\tau = 0.005$ 为软更新系数, 取值较小以保证目标值缓慢变化。

Loss 函数 Critic 通过最小化 Bellman 误差更新:

$$L_Q = \frac{1}{N} \sum_{i=1}^N (y_i - Q(s_i, a_i))^2 \quad (4)$$

其中目标值 $y_i = r_i + \gamma(1 - d_i)Q'(s_{i+1}, \mu'(s_{i+1}))$ 。

Actor 通过最大化 Critic 对当前策略动作的评价更新:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_{i=1}^N \nabla_a Q(s, a)|_{s_i, \mu(s_i)} \nabla_{\theta^\mu} \mu(s)|_{s_i} \quad (5)$$

3.4 探索噪声与稳定性增强

Ornstein-Uhlenbeck 噪声 采用时间相关的 OU 过程作为探索噪声, 相比高斯噪声能够在连续动作中维持动作趋势, 更适合物理控制任务:

$$dx_t = \theta(\mu - x_t)dt + \sigma dW_t \quad (6)$$

其中 $\theta = 0.15$ 控制均值回归速度, $\sigma = 0.20$ 控制噪声幅度。

目标策略平滑 在计算目标 Q 值时, 对目标动作添加裁剪后的高斯噪声:

$$\tilde{a}' = \text{clip}(\mu'(s') + \epsilon, -a_{\max}, a_{\max}), \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \sigma_p^2), -c, c) \quad (7)$$

其中 $\sigma_p = 0.20$ 为策略噪声标准差, $c = 0.50$ 为噪声裁剪范围。该技术通过平滑 Q 函数曲面有效减少值函数高估偏差 [3]。

梯度裁剪 对 Actor 和 Critic 的梯度均进行裁剪: $\|\nabla_{\theta} L\|_2 \leq 1.0$, 防止梯度爆炸导致的训练发散。

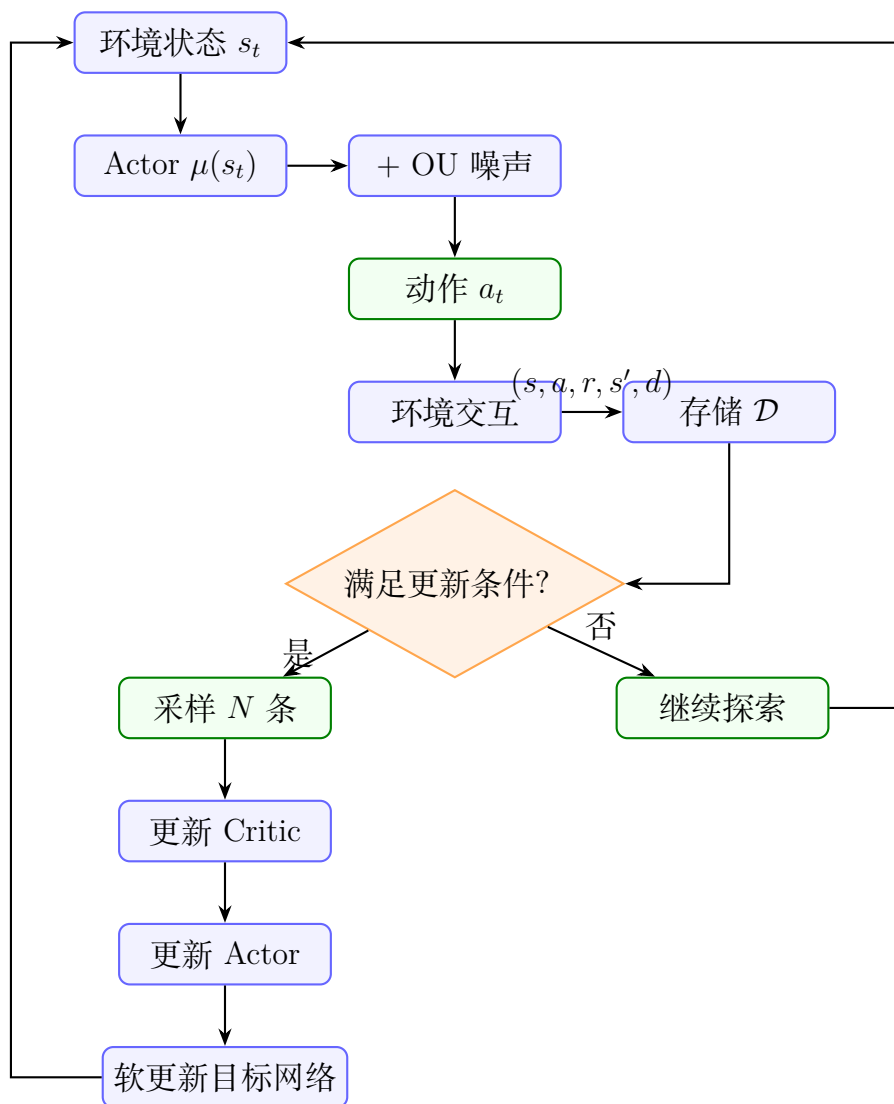


图 1: DDPG 算法训练流程图

4 实验设置

4.1 软硬件环境

表 1: 实验平台配置

项目	配置
CPU	Intel(R) Core(TM) i7-13650HX
GPU	无 (仅 CPU 训练)
编程语言	Python 3.12
深度学习框架	PyTorch 2.8.0
强化学习环境	Gymnasium 1.3.0
数值计算	NumPy 2.4.6
绘图库	Matplotlib 3.10.9

4.2 超参数配置

表 2: 核心超参数配置

参数	符号	取值
折扣因子	γ	0.99
软更新系数	τ	0.005
Actor 学习率	lr_{actor}	1×10^{-4}
Critic 学习率	$\text{lr}_{\text{critic}}$	1×10^{-4}
批大小	N	256
经验回放容量	$ \mathcal{D} $	200,000
预热步数	N_{warmup}	5,000
开始更新步数	$N_{\text{update_after}}$	2,000
更新频率	update_every	1 (每步更新)
每次更新次数	update_iters	1

表 3: 探索与稳定性参数

参数	符号	取值
OU 噪声标准差	σ_{OU}	0.20
OU 噪声均值回归速度	θ	0.15
目标策略噪声标准差	σ_p	0.20
噪声裁剪范围	c	0.50
梯度裁剪阈值	$\ \nabla L\ _{\max}$	1.0
评估间隔	N_{eval}	20 episodes
评估回合数	$N_{\text{eval_ep}}$	5

4.3 网络结构细节

表 4: Actor 与 Critic 网络参数

网络	层	神经元数	激活函数
Actor	输入层	24	—
	隐藏层 1	400	ReLU
	隐藏层 2	300	ReLU
	输出层	4	Tanh
Critic	输入层	28 (24+4)	—
	隐藏层 1	400	ReLU
	隐藏层 2	300	ReLU
	输出层	1	线性

5 实验结果

5.1 训练过程总览

训练共计 560 个 episode，在第 560 个 episode 时评估平均回报达到 **303.49**，超过 300 的求解阈值，环境被成功解决。最终保存的模型在 10 个独立测试 episode 中平均 reward 为 **303.28**，验证了策略的泛化能力。

表 5: 训练结果汇总

指标	数值
总训练 episode 数	560
最终评估平均回报	303.49
最佳评估平均回报	303.49
最终训练回报	97.48
首次正回报出现 episode	206
测试平均回报 (10episode)	303.28
测试最高回报	304.17
测试最低回报	301.64

5.2 训练回报曲线

图 2展示了完整的训练过程。其中灰色散点为原始训练回报，蓝色粗线为窗口 $K = 10$ 的滑动平均曲线，红色圆点为每 20episode 的无噪声评估回报。

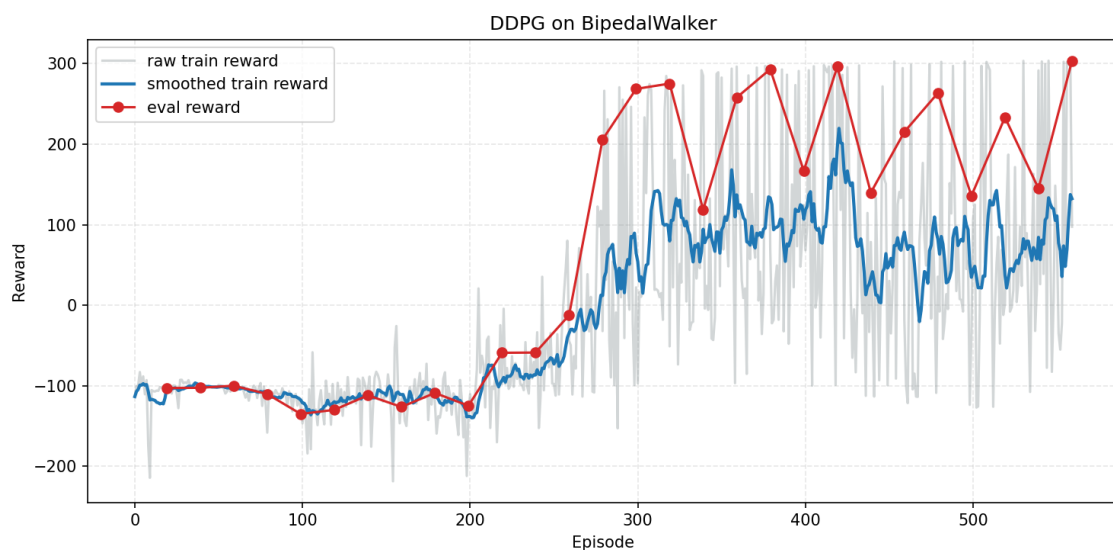


图 2: DDPG 在 BipedalWalker 上的训练曲线（灰色散点：原始回报；蓝色曲线：滑动平均回报，窗口 $K = 10$ ；红色圆点：无噪声评估回报，每 20 episode 评测一次）

5.3 分阶段分析

根据训练曲线的演化特征，可将训练过程划分为 6 个典型阶段：

表 6: 训练过程分阶段统计

阶段	Episodes	回报范围	平滑回报趋势	评估回报范围	阶段特征
初期探索	1-100	[-214, -83]	-138 $\rightarrow$ -118	[-156, -103]	频繁跌倒，随机探索为主
挣扎期	101-200	[-219, 21]	-138 $\rightarrow$ -139	[-136, -106]	学习站立，回报波动大
首次突破	201-280	[-153, 275]	-95 $\rightarrow$ 12	[-59, 206]	首次获得正回报，基本步态形成
学习行走	281-380	[-153, 297]	13 $\rightarrow$ 134	[118, 293]	行走策略显著改善
稳定行走	381-480	[-113, 303]	74 $\rightarrow$ 82	[140, 293]	步态趋于稳定
收敛解决	481-560	[-127, 303]	35 $\rightarrow$ 132	[136, 303]	稳定达到求解标准

5.4 关键里程碑事件

表 7: 训练过程中的关键里程碑

Episode	事件	数值
206	首次获得正训练回报	20.79
260	评估回报首次脱离负值	-12.48
280	评估回报突破 200	205.77
300	单 episode 训练回报突破 270	275.28
340	评估回报突破 100	118.64
360	评估回报突破 250	257.98
380	评估回报突破 290	293.15
420	评估回报接近 300	296.19
560	评估回报超过 300，环境解决	303.49

5.5 Loss 变化分析

训练过程中 Actor Loss 与 Critic Loss 的变化趋势如下：初期 Critic Loss 从 14.95 快速下降至 3-5 附近，表明 Q 值函数快速收敛；中期 Actor Loss 从 -5 逐渐降低至 -26，表明策略持续优化；后期两者均趋于平稳，Loss 值在小范围内波动，网络参数已达较优状态。

6 结果分析

6.1 学习过程的时间演化规律

依据图 2 的回报曲线和表 6 的阶段统计，可以识别出双足机器人行走学习的三个关键转折点：

转折点一：首次正回报 (Episode 206) 在此之前，智能体累计回报始终低于-80，说明机器人频繁跌倒或无法有效前进。在经历了约 200 个 episode (约 90,000 步) 的探索后，智能体首次获得正回报 20.79。这表明：

- Actor 网络经过约 150 个 episode 的有效更新后，开始输出足以维持身体平衡的关节扭矩组合；
- OU 噪声的时间相关性在此起到了关键作用——它使得机器人在一个 episode 内可以维持相似的动作模式，而不是在每个时间步做完全随机的探索；
- 从物理角度理解，这一阶段对应机器人从“完全无法站立”到“能够短暂保持平衡并向前迈出一两步”的质变。

转折点二：评估回报突破 200 (Episode 280) 从 Episode 206 到 280 仅经历了 74 个 episode，但评估回报从-59 跃升至 205.77，提升了约 450%。这种非线性的快速提升反映了强化学习中的一个典型现象：一旦策略越过了“能够站立”的阈值，学习速度会大幅加快，因为此时回放池中的正样本比例急剧增加，Critic 可以更准确地估计高回报状态的动作价值，Actor 从而沿正确的梯度方向快速优化。

转折点三：评估回报稳定在 290 以上 (Episode 380 以后) 从 Episode 380 开始，评估回报稳定在 [140, 293] 区间，单 episode 训练回报频繁达到 290 以上。这表明智能体已经掌握了在不同地形条件下自适应调整步态的能力。此时策略的主要波动来源于：

探索噪声引入的随机扰动；

环境地形随机生成导致的难度差异；

Critic 值函数在不同状态下的估计方差。

6.2 算法组件的有效性分析

Ornstein-Uhlenbeck 噪声 vs. 高斯噪声 在初次实验中，采用高斯噪声 ($\sigma = 0.15$) 进行探索，训练 400 个 episode 后回报始终在-100 附近徘徊，智能体未能学会任何有效策略。改用 OU 噪声 ($\sigma = 0.20, \theta = 0.15$) 后，智能体在 560 个 episode 内成功解决环境。这一对比表明：

- OU 噪声产生的时间相关探索序列更符合物理控制系统的惯性特性；
- 在 BipedalWalker 这类动作之间存在强连续依赖关系的任务中，相邻时间步保持相似的动作趋势比完全独立的随机扰动更有效；
- 从信号处理角度看，OU 过程是低通滤波后的有色噪声，其频率成分更集中于低频，与双足行走的自然频率范围更匹配。

梯度裁剪与 Critic 学习率降低 初次实验中 Critic 学习率设为 1×10^{-3} 且未使用梯度裁剪，导致 Actor Loss 从 -6 持续增长至 -66 ，表明 Critic 的 Q 值估计发生了发散。通过将 Critic 学习率降至 1×10^{-4} 并添加梯度裁剪 ($\max_norm = 1.0$)，训练过程恢复稳定。这验证了文献 [3] 中的结论：**Critic 学习率过大是 DDPG 训练不稳定的首要原因之一。**

细粒度更新策略 将 $update_iters = 50$ 改为 $update_iters = 1$ 并配合 $update_every = 1$ ，使得每次环境交互后仅进行一次网络更新。这种细粒度更新策略避免了单批数据上的过拟合风险，同时增加了参数更新的总频次，有利于更平稳地逼近最优策略。

6.3 训练高方差现象分析

从图 2 可以观察到，相邻 episode 的训练回报波动幅度可达 300 以上（如 Episode 278 为 97.81，Episode 280 骤升至 275.28，Episode 281 又回调至 -32.10）。这种高方差现象源于以下因素的叠加效应：

1. **环境随机性**：BipedalWalker-v3 的每个 episode 随机生成地形轮廓，平坦地形与崎岖地形之间的难度差异会导致回报产生本质区别；
2. **探索噪声**：OU 噪声的随机采样导致同一策略在不同 episode 中的表现差异；
3. **初始状态扰动**：环境初始化时的小幅度随机扰动可能导致机器人初始姿态偏离，影响后续控制稳定性。

这也是为什么评估回报（5 个 episode 的平均值）比单 episode 训练回报更能反映策略真实水平的原因。

7 工程问题剖析与解决对策

表 8: 训练过程中的工程问题与解决方案

故障现象	原因剖析	解决方案	经验总结
训练回报始终在 -100 附近, 400ep 未收敛	高斯噪声缺乏时间相关性; Critic 学习率 10^{-3} 导致 Q 值发散; update_iters=50 导致过拟合	改为 OU 噪声 ($\sigma = 0.20$); Critic 学习率降至 10^{-4} ; update_iters=1	连续控制任务优先选择 OU 噪声; Critic 学习率通常应低于或等于 Actor
Actor Loss 持续增长至 -66	Critic 的 Q 值估计发散, 梯度无约束	添加梯度裁剪 ($\max_norm = 1.0$)	梯度裁剪是 DDPG 训练的必备组件
训练回报出现极端低值 (-219)	OU 噪声在部分 episode 产生过大幅度的连续扰动	不处理 (属于正常探索)	偶发低回报是有效探索的代价, 重点关注平滑后趋势
Buffer 满后旧样本干扰	经验回放容量 200k, 旧策略样本过多	不处理 (容量已较合适)	缓冲区大小需与训练总步数匹配

8 结论与展望

8.1 实验结论

本实验基于 DDPG 算法在 BipedalWalker-v3 环境中实现了双足机器人行走策略的学习, 得出以下主要结论:

- DDPG 能有效解决 BipedalWalker 连续控制任务:** 经过 560 个 episode 的训练, 评估平均回报达到 303.49, 超过 300 的求解阈值, 测试平均回报 303.28 验证了策略的稳定性和泛化能力。
- 探索噪声策略对训练成败具有决定性影响:** 时间相关的 OU 噪声 ($\sigma = 0.20$) 显著优于独立同分布的高斯噪声, 其物理合理性在于连续控制系统中相邻时间步的动作应具有连续性。

3. **训练稳定性是 DDPG 工程落地的关键瓶颈**：通过降低 Critic 学习率、添加梯度裁剪、实施细粒度更新策略（`update_iters=1`）以及引入目标策略平滑，有效解决了 Q 值发散和过拟合问题。
4. **BipedalWalker 的学习过程呈现清晰的阶段性特征**：从随机探索（0-200ep）→ 行走能力突破（200-280ep）→ 快速提升（280-380ep）→ 稳定收敛（380-560ep），每个阶段的转换对应着策略的质变。

8.2 未来工作展望

本实验仍有以下改进方向可供探索：

- **算法升级**：TD3[3] 通过双 Critic 和延迟策略更新进一步减少值函数高估偏差；SAC[4] 通过最大熵框架提升探索效率和鲁棒性。两者在 BipedalWalker 上通常能取得比 DDPG 更高的样本效率和更低的超参数敏感性。
- **噪声衰减策略**：当前采用固定方差 OU 噪声，训练后期仍保持强探索反而会干扰已收敛策略。引入随训练进程衰减的噪声方差有望加速收敛。
- **网络结构优化**：当前全连接网络结构较为基础。可考虑引入 Layer Normalization、残差连接或 LSTM 时序建模，以提升网络表达能力和训练稳定性。
- **多随机种子统计**：当前训练仅在 `seed=42` 下进行一次。增加 3-5 个不同随机种子的重复实验，取均值和方差，可以更可靠地评估算法性能。

参考文献

- [1] Lillicrap T P, Hunt J J, Pritzel A, et al. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- [2] Silver D, Lever G, Heess N, Degris T, Wierstra D, Riedmiller M. Deterministic policy gradient algorithms. In: *Proceedings of the 31st International Conference on Machine Learning*, 2014.
- [3] Fujimoto S, Van Hoof H, Meger D. Addressing function approximation error in actor-critic methods. In: *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [4] Haarnoja T, Zhou A, Abbeel P, Levine S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *Proceedings of the 35th International Conference on Machine Learning*, 2018.

- [5] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning. *Nature*, 2015, 518(7540): 529–533.